

深圳大学实验报告

课程名称 机器学习

项目名称 实验五：聚类算法

学 院 计算机与软件学院

专 业 软件工程（腾班）

指导教师 赖志辉

报 告 人 洪子敬 学号 2022155033

实验时间 2024 年 5 月 6 日至 2024 年 5 月 12 日

实验报告提交时间 2024 年 5 月 9 日

教务处制

一、实验目的与要求

1. 简述 K-means 聚类算法的原理与算法过程。
2. 熟练掌握 K-means 聚类算法与结果的展示，并代码实现，做一个 2 维或三维空间中的 2~3 类点（每个类有 10 个点）聚类实验，把聚类结果用不同的颜色与符号表示。
3. 实现人脸图像（取前 2~3 个人的人脸图像）聚类实验与旋转物体（在 COIL20 数据集中取前 2~3 个类的图像），把聚类结果用不同的颜色与符号表示，并把对应的图像放在相应点的旁边，让人一眼看出结果对不对；同时列表给出其在不同数据库在不同 K 时的聚类精度。
4. 把课堂上我们讲课时说的模型及其优化过程写在实验最后的思考题部分。
5. 参考前人论文，开发设计一个全新的聚类算法，把关键信息写在报告中；详细的内容可以写成一个长文提交在“论文提交处”。（本点不作硬性要求，有能力者为之）

二、实验内容与方法

1. 简要介绍 K-means 原理与算法流程；
2. 用 matlab 实现 K-means 算法，并使用简单数据集进行测试，在此基础上编写 K-means ++ 算法，并用相同数据集进行测试，比较一下改进之处；
3. 在二维空间中分别用人脸数据集与旋转物体数据集上进行聚类，将聚类结果用不同颜色的点加以展示，同时把对应图像放在对应的点旁边，便于判断对错；
4. 在不同数据集上比较不同 K 时的聚类精度；
5. 完成之前课后给出的模型优化题；

三、实验步骤与过程

1. K-means (K 均值算法) 原理简介

(1) 聚类算法简单回顾

在无监督学习中，训练样本的标志信息是未知的，目标是通过对无标记训练样本的学习来揭示数据的内在性质和规律，为进一步的数据分析提供基础，此类任务研究最多、应用最多的是“聚类”。聚类是尝试将数据集中的样本划分为若干个通常是不相交的子集，每个子集均为一个“簇”，通过这样划分，每个簇可能对应于一个类别。

(2) 前提引入

K-means 有一个著名的解释：牧师-村民模型，具体内容如下所示：

有四个牧师去郊区布道，一开始牧师们随意选了几个布道点，并且把这几个布道点的情况公告给了郊区所有的村民，于是每个村民到离自己家最近的布道点去听课。

听课之后，大家觉得距离太远了，于是每个牧师统计了一下自己的课上所有的村民的地址，搬到了所有地址的中心地带，并且在海报上更新了自己的布道点的位置。

牧师每一次移动不可能离所有人都更近，有的人发现 A 牧师移动以后自己还不如去 B 牧师处听课更近，于是每个村民又去了离自己最近的布道点……；

就这样，牧师每个礼拜更新自己的位置，村民根据自己的情况选择布道点，最终稳定了下来我们可以发现该牧师的目的是为了让每个村民到其最近中心点的距离和最小。

这个模型也揭示了 K-means 算法的内涵所在。

(3) 算法介绍

给定样本集 $D = \{x_1, x_2, \dots, x_m\}$, K 均值算法针对聚类所得簇划分 $C = \{C_1, C_2, \dots, C_k\}$, 目标是为了最小化平方损失函数:

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

其中 $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 是簇的均值向量, 上式在一定程度上刻画了簇内样本围绕均值向量的紧密程度, E 值越小则簇内样本的相似度越大, 类别越正确。

下面是 k 均值算法的算法流程:

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇数 k .

过程:

- 1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$
- 2: **repeat**
- 3: 令 $C_i = \emptyset$ ($1 \leq i \leq k$)
- 4: **for** $j = 1, 2, \dots, m$ **do**
- 5: 计算样本 x_j 与各均值向量 μ_i ($1 \leq i \leq k$) 的距离: $d_{ji} = \|x_j - \mu_i\|_2$;
- 6: 根据距离最近的均值向量确定 x_j 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;
- 7: 将样本 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$;
- 8: **end for**
- 9: **for** $i = 1, 2, \dots, k$ **do**
- 10: 计算新均值向量: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$;
- 11: **if** $\mu'_i \neq \mu_i$ **then**
- 12: 将当前均值向量 μ_i 更新为 μ'_i
- 13: **else**
- 14: 保持当前均值向量不变
- 15: **end if**
- 16: **end for**
- 17: **until** 当前均值向量均未更新

输出: 簇划分 $C = \{C_1, C_2, \dots, C_k\}$

(4) 算法的优劣

优点: 容易理解, 聚类效果不错, 虽是局部最优但往往就够了;

处理大数据集时, 该算法可以保持较好的伸缩性;

当簇近似高斯分布的时候, 效果非常不错;

算法复杂度低, 且只需要调节参数 k ;

缺点: k 值需要人为设定, 不同 k 值得到的结果不一样;

对初始簇中心敏感, 不同的选取方式会得到不同的结果;

对异常值敏感;

样本只能归为一类, 不适合做多分类任务;

不适合太离散的分类、样本类别不平衡的分类、非凸形状的分类;

2. 改进算法 K-means++ 介绍

(1) 算法来源: 由于 k 个初始化的质心的位置选择对最后的聚类结果和运行时间都有很大的影响, 因此需要选择合适的 k 个质心。如果仅仅是完全随机的选择, 有可能导致算法收敛很慢。K-Means++ 算法就是对 K-Means 随机初始化质心的方法的优化。

(2) K-means++ 算法流程 (初始化质心的方法) 如下:

① 从输入的数据点集合中随机选择一个点作为第一个聚类的中心 μ_1 ;

② 对于数据集中的每个点 x_i , 计算它与已选择聚类中心中距离最近聚类中心的距离 $D(x_i)$,

$$D(x_i) = \arg \min \|x_i - \mu_r\|_2^2, \text{ 其中 } r = 1, 2, 3, \dots, k(\text{selected})$$

- ③选择一个新的数据点作为新的聚类中心，选择的的原则是：
 $D(x)$ 较大的点，作为聚类中心的概率越大，即选择最远的点来做聚类中心；
- ④重复 2、3 步直到选择 k 个质心；
- ⑤利用这 k 个质心作为初始质心去运行标准的 K-means 算法；

3. K-means 的简单编码实现 (matlab)

(1) 实现思想：按照 K-means 前面的算法流程，首先随机选择结果点作为中心点，计算每个点距离中心点的距离，将每个样本点归于离自己最近的样本点的一簇；接着重新计算更新每个簇的中心点；之后循环往复上面步骤，直到样本中心点不再移动。

(2) 代码实现及测试

下面为 K-means 代码的基本实现，

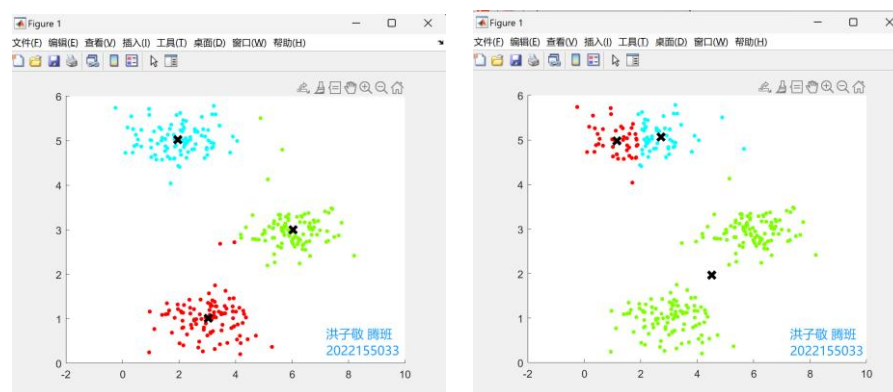
```
function [centroids, label]=Kmeans(X, k, initial_centroids)
% initial
centroids=initial_centroids;
history=[];
previous_centroids=initial_centroids;
[m, ~]=size(X);
label=zeros(m, 1);
steps=0;
% iterative update
while(true)
    previous_centroids=centroids;
    steps=steps+1;
    label=zeros(m, 1);
    for i=1:m
        dist=zeros(k, 1);
        Xi=X(i, :);
        for j=1:k
            dist(j)=sum((Xi-centroids(j, :)).^2);
        end
        [~, index]=sort(dist);
        label(i)=index(1);
    end

    % compute new centroids
    for i=1:k
        Xi=X(label==i, :);
        miu=mean(Xi, 1);
        centroids(i, :)=miu;
    end

    if(centroids==previous_centroids)
        break;
    end

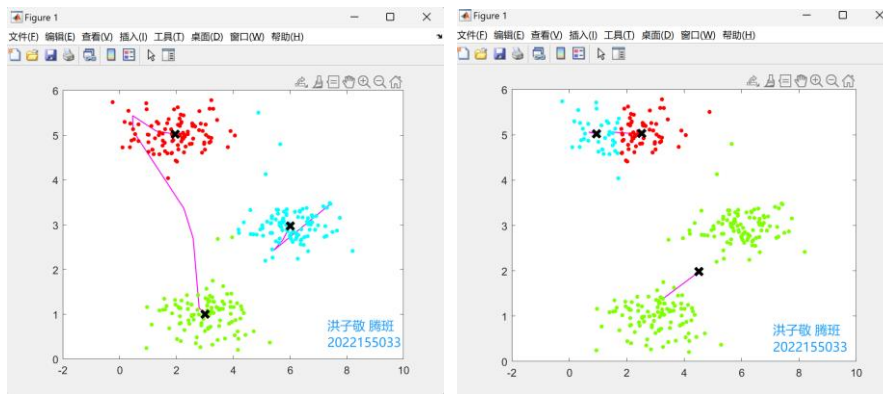
    history=[history;previous_centroids];
end
```

这里为了测试其效果，我从网上下载了一个典型聚类数据集 ex7data2 来测试其效果，数据集大体可以看成三类，我们设置 k 为 3 进行实验，实验结果如下所示：



我们可以看到聚类的结果有两种，很明显左边聚类的效果更好。出现此情况的原因主要与初始点的选取有关，右边是初始点选取的不是很好导致的。

为了进一步探寻这两种情况的中心点在迭代过程的变化，我将每次中心点记录下来，在迭代完成之后，把所有点连接起来就构成了中心点的变化轨迹图。对应于上面两种结果，下面展示随机中心点下的一种中心点变化图：



4. K-means++的简单编码实现（matlab）

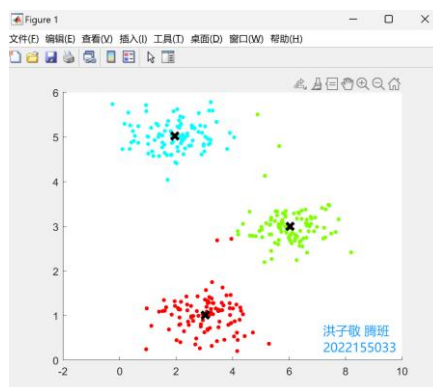
（1）实现思想： 首先随机选取样本作为中心点，接着计算其他点距离中心点的距离，选择距离最大的点加入中心点；接着继续计算其他点离这些中心点距离和最大的加入中心点；重复此过程，知道选到足够的中心点为止。

（2）代码实现及测试

K-means 主要是针对初始点的选取，所以我们先用 K-means++选好初始点再使用 K-means 进行聚类即可。下面左图为其代码演示，我们同样用此数据集进行聚类可以得到右图结果：

```
function [initial_centroids]=KmeansPlusPlus(X,k)
randix=randperm(size(X,1));
initial_centroids=X(randix(1,:),:);
centroids_num=1;
[m,~]=size(X);
while(centroids_num<k)
    max_dist=0;
    index=0;
    for i=1:m
        Xi=X(i,:);
        dist=0;
        for j=1:centroids_num
            dist=dist+sum((Xi-initial_centroids(j,:)).^2);
        end
        if(dist>max_dist)
            max_dist=dist;
            index=i;
        end
    end
    initial_centroids=[initial_centroids;X(index,:)];
    centroids_num=centroids_num+1;
end
```

洪子敬 腾讯
2022155033



我们可以发现，用 K-means++后的聚类结果基本固定为之前出现的聚类结果中较好的那种。

5. 人脸数据集和旋转物体数据集在二维空间上的聚类

（1）在 AR_Gray_50y40 人脸数据集上，取前三类每类 10 张人脸图像。先用 PCA 进行降至 2 维，再用 K-means 算法进行聚类，并聚类结果用不同颜色的点展示，为了方便分辨对错，我附上了样本点对应的人脸图像在其旁边。聚类结果如图（a）所示：

（2）在 Coil20 旋转物体数据集上，也是取前三类各 10 张图像。也是用 PCA 降至 2 维，再用 K-means 算法进行聚类，并聚类结果用不同颜色的点展示，也附上了样本点对应的旋转物体图像在其旁边。聚类结果如图（b）所示：

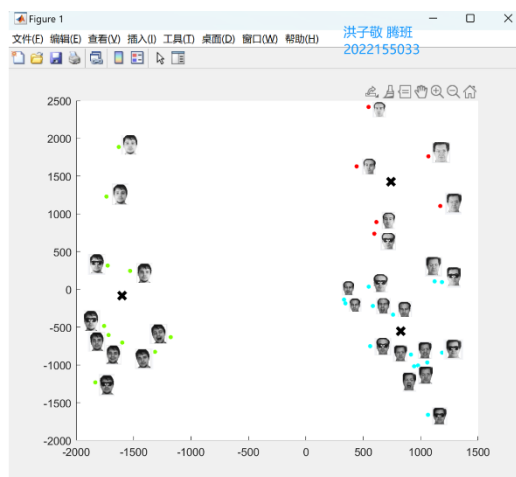


图 (a)

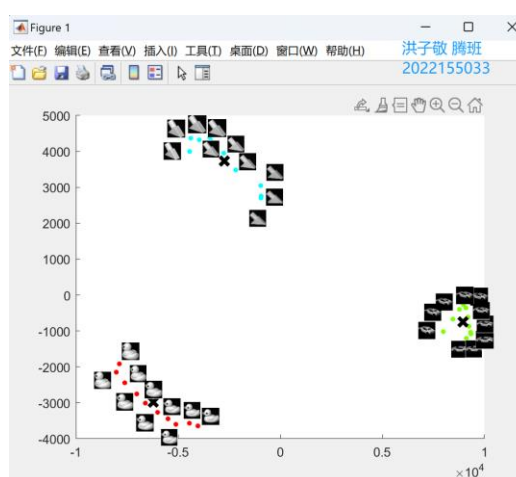


图 (b)

观察实验结果可知，在人脸数据集上聚类的结果并不是很好，左边那一类分的很正确，可右边两类却混在了一起。这可能与使用 PCA 降维有关，右边两类提取到的特征比较相近，不是很难分开。而在旋转物体数据集上，聚类的效果明显很好，这与 PCA 提取的特征有关，特征都非常明显地进行区分开。除此之外，我们还发现，聚类还保持了各类的流型结构，按照一个方向对图片进行旋转。

6. 在不同数据集上计算不同 K 时聚类精度

聚类精度的计算方法：首先正确将各个中心点的标签找到（用 K 近邻的思想进行标注），找到后，将同一簇的每个点标注为中心点的标签，最后将每个点的标注标签和其正确的标签进行比较，统计正确数量，计算正确率即得到聚类精度。

下面为 matlab 实现的精度计算代码实现：

```
% calculate correct rate
% first ensure the label of each cluster
clusterClasses=zeros(k,1);
for i=1:k
    cnt=zeros(class_num,1);
    label_i=Label_train(label==i);
    for j=1:size(label_i,2)
        cnt(label_i(j),1)=cnt(label_i(j),1)+1;
    end
    [~,index]=sort(cnt,"descend");
    clusterClasses(i)=index(1);
end
% second calculate the correct num
right=0;
for i=1:k
    label_i=Label_train(label==i);
    for j=1:size(label_i,2)
        if(label_i(j)==clusterClasses(i))
            right=right+1;
        end
    end
end
% finally calculate the correct rate
rate=right/(class_num*train_num);
fprintf("The K-means accuracy is : %f\n",rate);
```

洪子敬 腾班
2022155033

这里我们采用了 AR_Gray_50by40、ORL56_46、Yale_face10080、coil-20-proc 四个数据集上进行实验，均是取前四类，每类 10 张图片进行实验，同时 K 从 2 到 7 进行选取。此外，这里我们不用 PCA 进行降维再聚类了，因为这样会损失特征，会使聚类精度下降。实验结果如下所示：（由于聚类结果有多多样性，这里是计算多次，取最好的一次）

k的不同取值	2	3	4	5	6	7
AR_Gray_50by40	0.5	0.75	0.975	0.725	0.925	0.925
ORL56_46	0.5	0.725	0.95	0.975	0.95	0.9
coil-20-proc	0.5	0.75	1	1	1	1
Yale_face10080	0.45	0.65	0.7	0.8	0.925	0.95

根据实验我们可以发现，对于 AR_Gray_50by40 数据集，当 K=4 时效果最好；对于 ORL56_46

数据集，当 K=5 时效果最好；对于 Yale_face10080 数据集，当 K=7 效果最好；对于 coil-20-proc 数据集，由于不同类别之间样本点的特征差异较大，所以聚类效果都比较好，在 K 为 4、5、6、7 时精度都是最高的。

四、思考题

$$1. \min_{B,P} \sum_i \|x_i - BP^T x_i\|_2^2 + \lambda \|P\|_F^2 \quad s.t. B^T B = I$$

解：上述优化问题可以化为

$$\begin{aligned} & \min_{B,P} \text{tr} \left(\sum_i (x_i - BP^T x_i)^T (x_i - BP^T x_i) + \Lambda P^T P \right) \\ &= \min_{B,P} \text{tr} \left(\sum_i (x_i^T x_i - 2x_i^T BP^T x_i + x_i^T PB^T BP^T x_i) + \Lambda P^T P \right) \\ &= \min_{B,P} \text{tr} (X^T X - 2X^T BP^T X + X^T PB^T BP^T X + \Lambda P^T P) \end{aligned}$$

由于 $B^T B = I$ 上式可以化为

$$\min_{B,P} \text{tr} (X^T X - 2X^T BP^T X + X^T PP^T X + \Lambda P^T P)$$

对 B 而言，上式中只有 $-2X^T BP^T X$ 对其才有影响

由右边定理可知，对 $P^T XX^T$ 进行奇异值分解，我们可以得到 $B = UV^T$ ；

求解出 B 之后，我们让上式对 P 求偏导并令其偏导为 0，可以得到下式

$$-2XX^T B + 2XX^T P + 2P\Lambda = 0$$

上式只有 P 未知，因而我们可以比较轻松地求解出 P ；

因此，此优化问题的求解过程应该是个迭代过程，首先奇异值分解求解出 B ，再带入上式求解 P ，循环往复，不断迭代，直到 B 、 P 达到稳定（变化小于一个阈值）；

Theorem 1 [5]: Let S and Z be $c \times k$ matrices and Z has rank k . Given the following optimization problem

$$\hat{S} = \arg \max_S \text{Tr}(S^T Z) \quad s.t. \quad S^T S = I_k$$

Suppose the SVD of Z is $Z = \tilde{U} \tilde{D} \tilde{V}^T$, then $\hat{S} = \tilde{U} \tilde{V}^T$.

From Theorem 1, we can know that for given B and h in (12), suppose the SVD of $(h1^T - Y^T)DXB$ is $(h1^T - Y^T)DXB = U \tilde{D} V^T$, then

$$A = UV^T \quad (13)$$

$$2. \min_{B,P} \sum_{i,j} \|x_i - BP^T x_j\|_2^2 W_{ij} + \lambda \|P\|_F^2 \quad s.t. B^T B = I$$

解：上述优化问题可以化为

$$\begin{aligned} & \min_{B,P} \text{tr} \left(\sum_{i,j} (x_i - BP^T x_j)^T (x_i - BP^T x_j) W_{ij} + \Lambda P^T P \right) \\ &= \min_{B,P} \text{tr} (\sum_i \sum_j (x_i^T x_i - 2x_i^T BP^T x_j + x_i^T PB^T BP^T x_i) W_{ij} + \Lambda P^T P) \\ &= \min_{B,P} \text{tr} (\sum_i \sum_j (x_i^T x_i - 2x_i^T BP^T x_j + BP^T x_i x_i^T PB^T) W_{ij} + \Lambda P^T P) \\ &= \min_{B,P} \text{tr} (X^T DX - 2X^T WBP^T X + BP^T XDX^T PB^T + \Lambda P^T P) \end{aligned}$$

对 B 而言，上式中只有 $-2X^T WBP^T X + BP^T XDX^T PB^T$ 含有 B ，对其有影响，但由于

$$\text{tr}(BP^T XDX^T PB^T) = \text{tr}(B^T BP^T XDX^T P) = \text{tr}(P^T XDX^T P)$$

此时 B 只剩前面一项，回到和前者的优化问题一样，此时我们同理只需要将 $P^T XX^T W$ 进行奇异值分解即可得到 $B = UV^T$ ；

接着同理对上式的 P 进行求偏导并令其为 0，可以得到

$$XDX^T P B^T B + P \Lambda = X X^T W B$$

上式只有 P 未知，因而在求解出 B 后，我们可以比较轻松地求解出 P ；
因此，此优化问题的求解过程也是个迭代过程，首先奇异值分解求解出 B ，再带入上式求解 P ，循环往复，不断迭代，直至 B 、 P 达到稳定（变化小于一个阈值）；

$$3. \max_{\Theta, \Phi} m^{-1} \|Y\Theta - X\Phi\|_2^2 + \lambda \|s_b^{\frac{1}{2}} \Phi\|_F^2 \quad s.t. \quad m^{-1} \|Y\Theta\|_F^2 = 1$$

解：上述优化问题可以转化为：

$$\max_{B, \Phi} tr(m^{-1}(Y\Theta - X\Phi)^T(Y\Theta - X\Phi) + \Lambda \left(s_b^{\frac{1}{2}} \Phi\right)^T \left(s_b^{\frac{1}{2}} \Phi\right)$$

利用拉格朗日乘数法加上限制项可以化为：

$$\begin{aligned} & \max_{B, \Phi} tr(m^{-1}(Y\Theta - X\Phi)^T(Y\Theta - X\Phi) + \Lambda \left(s_b^{\frac{1}{2}} \Phi\right)^T \left(s_b^{\frac{1}{2}} \Phi\right) + \beta(m^{-1} \|Y\Theta\|_F^2 - 1) \\ & = \max_{B, \Phi} tr(m^{-1}(\Theta^T Y^T Y \Theta - 2\Theta^T Y^T X \Phi + \Phi^T X^T X \Phi) + \Lambda \Phi^T s_b \Phi + \beta(m^{-1} \Theta^T Y^T Y \Theta - 1)) \end{aligned}$$

上式对 Φ 求导并令其为 0 可得：

$$m^{-1}(-2X^T Y \Theta + 2X^T X \Phi) + 2s_b \Phi \Lambda = 0$$

即：

$$ms_b \Phi \Lambda - X^T Y \Theta = X^T X \Phi \quad (1)$$

上式对 Θ 求导并令其为 0 可得：

$$m^{-1}(2Y^T Y \Theta - 2Y^T X \Phi) + 2\beta m^{-1} Y^T Y \Theta = 0$$

解得

$$\Theta = ((\beta + I)Y^T Y)^{-1} Y^T X \Phi \quad (2)$$

由此我们可以知道此优化模型的算法是每次随机初始化 Φ ，再带入②式解出 Θ ；接着再带入①式解出 Φ 并更新 Φ ，依此循环往复，直至 Φ 与 Θ 保持稳定（变化小于一个阈值）；

Ps：还有一题与题 2 基本一致，只换了变量名称，这里不加展示。

五、实验结论或体会

实验结论：

本次实验成功设计了 K-means 算法，还实现了其拓展 K-means++，同时在人脸数据集和旋转物体数据集上进行了聚类，并实现了对应的图像二维可视化；此外，还在不同数据集上测试不同的 K 的聚类精度大小，找到了各个数据集的最佳簇的个数；本次实验圆满结束。

实验体会：

本实验最大的收获就是进一步加深了对 K-means 算法及其拓展 K-means++ 的理解，看到了其在处理线性数据的可行性和高效性。之前对 K-means 的理解只停留于表面，现在自己编写代码才知道自己与之前的理解还是有所偏差，比如这个聚类精度的计算，之前都不知道原来 K 可以比类的个数多，也是扩展了自己的视野和收获其背后的原理知识；

此外，本实验采用了 matlab 编程，在编写可视化图像时，我发现了一些 matlab

自带的好用的函数（比如调色板等）可以帮助我节省很多时间，画出的图像也能够更加美观，也不用我手动去调色了；

另外实验过程中也因为学业繁忙读的论文不是很多，没有去思考创新的聚类算法，同时所有的聚类的算法并没有自己实现一遍，这也算是一个遗憾吧，等有时间了，一定要全部实现一次。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。